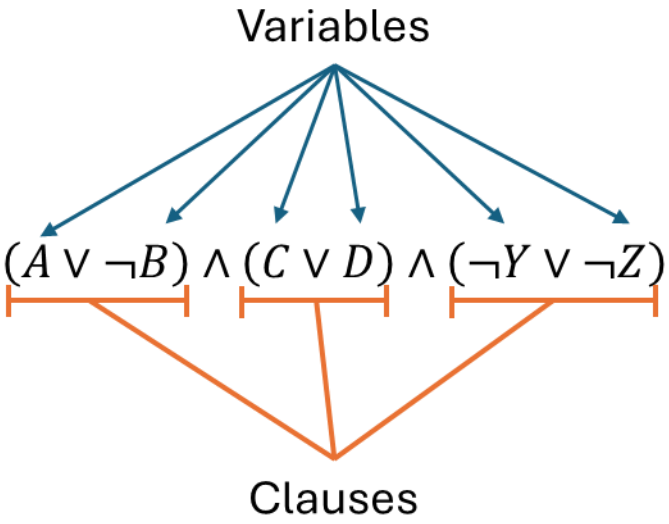
# COMP2001 Framework API

## Introduction

This framework is a Java framework designed for the teaching of heuristic decision support/AI methods. It includes the MAX-SAT problem domain as an example of an NP-hard optimisation problem. A MAX-SAT problem is defined as a Boolean formula in conjunctive normal form (CNF) which is a conjunction of disjunctive *clauses*. Each clause itself contains one or more *variables*. An example instance of a MAX-SAT problem could be $(A \vee \neg B) \wedge (C \vee D) \wedge (\neg Y \vee \neg Z)$.




A solution is represented by a binary string (sequence of bits) where each bit represents a truth value of each variable. In the above example, we have the variables $\{A, B, C, D, Y, Z\}$. A solution to this could be represented as $011011$ meaning that $B, C, Y,$ and $Z$ are assigned to be *true*, and $A,$ and $D$ to *false*.

The objective for MAX-SAT is to maximise the number of satisfied clauses. This framework aims to minimise the objective value; hence a modified objective function is used which is to minimise the number of unsatisfied clauses.

## API Specification

Below are all the classes and methods that you will need to use in the computing exercises. The classes and methods included in the LabRunner and associated weekly exercises are documented in the code itself.

The codebase follows the following house style rules in order:

- Names of member variables are prefixed “m_”.

- Names of arrays are prefixed ‘a’.
- Names of primitive types are prefixed with the first character of that type, for example an int will be prefixed ‘i’.
- Names of objects are prefixed with ‘o’.

For example, an array of ints that I want to call results that is a member variable of a class would be named “m_aiResults”.

## SAT

`public boolean hasEvaluationLimitExpired()`: returns *true* if the termination criterion has lapsed signalling to the search methods that they should stop exploring other solutions.

`public void copySolution(int iOriginIndex, int iDestinationIndex)`: creates a deep copy of the solution stored in memory index *iOriginIndex* and stores it in memory index *iDestinationIndex*.

`public int getObjectiveFunctionValue(int iSolutionIndex)`: evaluates and returns the objective value of the solution stored in memory index *iSolutionIndex*.

`public void bitFlip(int iBitIndex, int iMemoryIndex)`: flips the truth value of the variable with ID *iBitIndex* of the solution stored in memory index *iMemoryIndex*.

`public void exchangeBits(int iSolutionMemoryIndexA, int iSolutionMemoryIndexB, int iVariableIndex)`: exchanges the truth values of the variable with ID *iVariableIndex* between the two solutions in memory indices *iSolutionMemoryIndexA* and *iSolutionMemoryIndexB*.

`public int getBestSolutionValue()`: returns the objective value of the best solution found during the search.

`public void createRandomSolution(int iSolutionIndexToStore)`: initialises a new solution to the current MAX-SAT problem assigning truth values randomly. The solution is stored in memory index *iSolutionIndexToStore*.

`public int getNumberOfVariables()`: returns the number of variables present in the current MAX-SAT problem instance.

`public int getNumberOfMemes()`: returns the total number of memes associated with the solutions. Note – this functionality will not be used until computing exercise 6.

`public Meme getMeme(int iSolutionIndex, int iMemeNumber)`: returns the *iMemeNumber* indexed Meme object associated with the solution stored in memory index *iSolutionIndex*.

## Meme

`public int getMemeOption()`: returns the option currently expressed by the meme.

`public boolean setMemeOption(int iOption)`: changes the option currently expressed by the meme to *iOption* and returns *true* unless the option is out of range in which case the option will not be changed and *false* returned.

`public int getTotalOptions()`: returns the total number of different options this meme is able to express. If the total options are for example 3, then the valid options are 0, 1, and 2.

## ArrayMethods

`public static ArrayList<Integer> shuffle(ArrayList<Integer> oInts, Random oRandom)`: creates a copy of the ArrayList *oInts* and shuffles it using the supplied random number generator *oRandom* to ensure repeatability.

`public static int[] shuffle(int[] aiArray, Random oRandom)`: creates a copy of the integer array *aiArray* and shuffles it using the Fisher-Yates Shuffle Algorithm using the supplied random number generator *oRandom* to ensure repeatability.

## SATHeuristic

### Global Variables

`final Random m_oRandom` : the seeded random number generator that should be used by any subclass when making stochastic decisions.

`final int CURRENT_SOLUTION_INDEX` : the memory index where the solution being operated on should be stored.

`final int BACKUP_SOLUTION_INDEX`: the memory index where a backup of the solution should be stored before changing the current solution.

### Methods

`public void applyHeuristic(SAT oProblem)`: applies the heuristic to the solution of the problem *oProblem* stored in memory index `CURRENT_SOLUTION_INDEX`. Note this calls the below abstract method.

`public abstract void applyHeuristic(SAT oProblem, int iSolutionIndex)`: applies the heuristic to the solution of the problem *oProblem* stored in memory index *iSolutionIndex*.

`protected Random getRandom()`: returns the seeded random number generator that subclasses should use when making stochastic decisions. Note the member variable `m_oRandom` can be accessed directly based upon your preference.

## SinglePointSearchMethod

Creates a search method with a population size of 2; one for the current solution, and one for the backup solution. Note that when an instance of a *SinglePointSearchMethod* is created, the solution in the current solution index is copied to the backup solution index.

### Global Variables

`final SAT m_oProblem` : a reference to the problem (and instance) being solved.

`final Random m_oRandom` : the seeded random number generator which should be used by all search methods when making stochastic decisions.

`final int CURRENT_SOLUTION_INDEX` : the memory index where the solution being operated on is stored.

`final int BACKUP_SOLUTION_INDEX` : the memory index where a backup of the solution should be stored before changing the current solution.

### Methods

`public int run()` : calls the *runMainLoop* method, to be implemented by all sub-classes, and then returns the objective value of the solution stored in index `CURRENT_SOLUTION_INDEX`.

`protected abstract void runMainLoop()` : to be implemented by all sub-classes and defines a single pass of the single point-based search method. Note that the while loop of each search method has been lifted to the LabRunner Classes to allow for data collection. The responsibility of this loop is therefore to apply the logic for each iteration of the search method *and to ensure that the accepted solution is stored in the CURRENT_SOLUTION_INDEX*.

## PopulationBasedSearchMethod

Creates a search method with a specified population size. Internally, there are $2 \times POPULATION\_SIZE + 1$ memory indices for storing solutions using the following memory layout:

- $[0, \dots, POPULATION\_SIZE - 1]$ are used for storing the current/accepted population of solutions.
- $[POPULATION\_SIZE, \dots, 2 \times POPULATION\_SIZE - 1]$ are used for storing intermediate solutions/offspring.
- $[2 \times POPULATION\_SIZE]$ is used for storing a backup solution, if required.

### Global Variables

`final SAT m_oProblem` : a reference to the problem (and instance) being solved.

`final Random m_oRandom` : the seeded random number generator which should be used by all search methods when making stochastic decisions.

`final int POPULATION_SIZE` : the size of the population. Note the memory layout detailed above.

`final int BACKUP_SOLUTION_INDEX` : the memory index where a backup solution can be stored if needed.

### Methods

`public int run()` : calls the *runMainLoop* method, to be implemented by all sub-classes, and then returns the objective value of the best solution found during this

iteration/generation of the population-based search algorithm as stored in memory indices $[0, \dots, POPULATION\_SIZE - 1]$.

`protected abstract void runMainLoop()` : to be implemented by all sub-classes and defines a single pass of the single point-based search method. Note that the while loop of each search method has been lifted to the LabRunner Classes to allow for data collection. The responsibility of this loop is therefore to apply the logic for each iteration of the search method *and to ensure that the accepted population is stored in the memory indices* $[0, \dots, POPULATION\_SIZE - 1]$.